COMP4135 SEM Weekly Report 2

Group 2 (Tactical Cards)

Team members

Guangjun HU (20511787)
Hongshuo HU (20808603)
Cheng YE (20513323)
Zehan CHEN (20807750)

School of Computer Science
University of Nottingham

Submission Date: 03/04/2026

Contents

1 Specification and Design Decisions **2**
1.1 Requirements Validation . . . 2
1.1.1 Why WR2 Still Follows Requirements Engineering . . . 3
1.1.2 Updated Functional and Nonfunctional Priorities . . . 4
1.1.3 Key Design Trade-offs . . . 4
1.2 Architectural/System Diagram . . . 4

2 Prototyping Process **6**
2.1 Prototype . . . 6
2.1.1 Gameplay and Feature Walkthrough . . . 8
2.2 Design Validation . . . 9

3 Implementation Strategy **11**
3.1 Code Structure . . . 11
3.1.1 Runtime Responsibilities Beyond the Diagram . . . 12
3.2 Critical Implementation Choices . . . 13

4 Summary and Reflections **15**
4.1 Team Members and the Division of Labor . . . 15
4.2 Completed Process . . . 15
4.3 Prioritised Tasks . . . 16
4.3.1 P0 Submission-Critical Tasks . . . 16
4.3.2 P1 Strongly Recommended Tasks . . . 16
4.3.3 P2 Deferred or Externally Blocked Tasks . . . 16
4.4 Future Plan . . . 16

Reference **20**

A Meeting Minutes and Working Records **21**
A.1 Recovered Checkpoint (15/03/2026) . . . 21
A.2 WR1 Kickoff and Alignment Meetings . . . 22
A.2.1 Meeting 1 (17/03/2026) . . . 22
A.2.2 Meeting 2 (18/03/2026) . . . 22
A.3 WR2 Evidence Freeze Note . . . 23
A.4 Supporting Record Index . . . 23
A.5 Traceability and Priority Sources . . . 24
A.6 Retrospective Coordination Board . . . 25

Chapter 1

Specification and Design Decisions

This chapter explains how Tactical Cards moved from the WR1 requirements baseline into a narrower Unity prototype. The focus is traceability rather than redesign: what WR2 inherits, what it simplifies, and what it defers so the prototype remains demonstrable.

1.1 Requirements Validation

Table 1.1 makes the WR1 to WR2 relationship explicit. It distinguishes direct inheritance, implementation-driven narrowing, and visible deferred work.

Table 1.1: WR1 to WR2 requirements traceability

WR1 baseline	WR2 status	Reason for change	Evidence source	RE conformance
WR1 personas and first-battle scenarios for casual strategy players.	**Inherited**	WR2 still validates the same audience and first-use situations instead of redefining them.	WR1 report; walkthrough note; prototype screenshots.	Yes. Validation still follows the user-facing concerns elicited in WR1.
Core battle loop: turn start, draw, unit/card/target selection, enemy turn, and battle check.	**Inherited**	This is the minimum playable slice needed to demonstrate the WR1 concept without widening scope.	Snapshot verification; first-pass script note; runtime scene.	Yes. WR2 implements the same central interaction flow and checks it against runtime evidence.
6x6 board, battle UI, hand zone, turn controls, and starter cards `Move`, `Strike`, `Guard`, and `Push`.	**Inherited**	The starter content stayed narrow so the tactics loop remained readable and testable within coursework time.	Snapshot verification; architecture constraints note; Figures 2.2–2.4.	Yes. WR2 preserves the functional baseline promised for the demo combat scenario.

WR1 baseline	WR2 status	Reason for change	Evidence source	RE conformance
Readable feedback, onboarding cues, and understandable turn results as non-functional usability goals.	**Modified**	WR2 keeps prompts, highlights, HP visibility, guard state, logs, and text outcomes, but tutorial depth and presentation polish were reduced for stability.	Walkthrough note; EditMode test note; merged PR #5.	Yes, in partial form. The simplification is explicit rather than hidden.
WR1 system model and architecture expectations around turn flow, grid logic, units, cards, and UI.	**Modified**	The responsibility split stayed, but implementation was narrowed to one battle scene, one shared `UnitController`, one explicit `CardResolver`, deterministic `EnemyAI`, and a prototype-first UI path.	Architecture constraints note; first-pass scripts; implementation decisions; Figures 1.1 and 3.1.	Yes. The architecture remains traceable to WR1 and the scope-control decisions are named explicitly.
Organisational requirement for repository-based teamwork, traceable records, and demonstrable project management.	**Modified**	Earlier collaboration used mixed tools, so WR2 closes out on a cleaned GitHub PR workflow while still labelling weaker board evidence as blocked or retrospective.	Collaboration history; WR2 todo; task-board evidence note; PRs #6, #7, #8, #9, #12, #13, #15.	Partially. WR2 now evidences the repository workflow honestly while separating it from weaker board evidence.
Broader concept growth beyond the frozen demo, including larger card pools, multi-scene flow, stronger playtesting, and richer battle presentation.	**Deferred**	These items would widen the prototype beyond the evidencable WR2 vertical slice and weaken the report's claims.	WR2 todo; architecture constraints note; WR3 prep log; walkthrough findings.	Yes. Deferred items remain visible backlog work rather than silent omissions.

1.1.1 Why WR2 Still Follows Requirements Engineering

WR2 still follows the WR1 requirements engineering process for three reasons:

- It keeps the same audience, stakeholder concerns, and validation scenarios as the baseline for implementation review.
- Every implementation-driven change is justified through a named scope-control source, especially the architecture constraints note and the WR2 implementation decision log.
- Any incomplete item stays labelled as **partial** or **deferred** and is carried into the prioritised task list in Chapter 4 instead of being silently dropped.

WR2 is therefore not a second independent requirements exercise. It is a traceable reduction of WR1 into a playable vertical slice whose compromises remain explicit and evidence-backed.

1.1.2 Updated Functional and Nonfunctional Priorities

- **Functional priority.** Keep one readable starter set: draw three cards, allow up to two plays per turn, and resolve only `Move`, `Strike`, `Guard`, and `Push`.
- **Non-functional priority.** Keep hit points, guard state, prompts, and battle-end text visible, while treating stronger onboarding and richer presentation as later work.

1.1.3 Key Design Trade-offs

The main design trade-off in WR2 was scope versus stability. A larger deck, more combat roles, and more complex effects would increase variety, but they would also increase balancing work and testing risk. The implemented choice was to keep the starter deck small, keep enemy behaviour simple, and keep script responsibilities easy to trace. This makes the prototype easier to demonstrate and easier to connect back to the WR1 requirements. The resulting WR2 scope is smaller than the broader direction proposed in WR1, but it is clearer and more defensible as a playable prototype.

1.2 Architectural/System Diagram

The current architecture follows a single-battle structure. Figure 1.1 shows the runtime modules used by the demo and how scene managers, UI, cards, grid, and units interact during play.

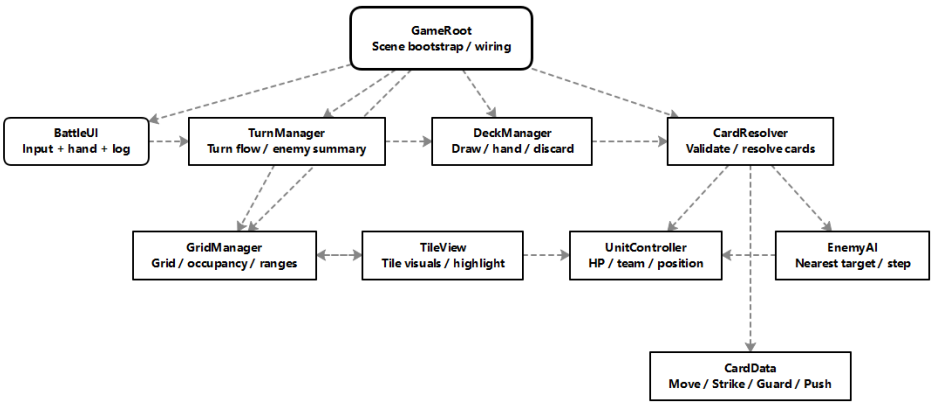

Figure 1.1: Current WR2 runtime architecture for the Tactical Cards prototype

At runtime, `GameRoot` wires the battle scene, `TurnManager` owns phase changes and card-play limits, and `BattleUI` collects player input. `DeckManager` controls draw, hand, and discard state, `CardResolver` resolves the four card types, `GridManager` owns tiles and reachability, and `UnitController` stores shared unit state. This is the full module set needed by the narrowed WR2 slice.

The WR1 to WR2 architecture carry-over can be summarised more explicitly as follows:

- **Inherited.** Separate responsibilities for turn flow, grid logic, cards, units, and UI still provide the clearest high-level structure for the demo.
- **Modified for scope control.** The architecture now assumes one self-contained battle scene instead of a wider scene or campaign structure.
- **Modified for traceability.** Player and enemy runtime state were collapsed into one shared `UnitController` with `TeamType`, which keeps health, guarding, and occupancy rules easier to explain.
- **Modified for implementation speed.** Card behaviour is resolved in one explicit `CardResolver` rather than a generic effect hierarchy, matching the four-card WR2 scope.
- **Modified for validation clarity.** `EnemyAI` stays deterministic and shallow so enemy turns are readable in demonstration and easier to verify against screenshots and logs.
- **Modified for prototype practicality.** `BattleUI` uses a lightweight panel plus raycast input path, prioritising clarity and integration speed over a more polished UI stack.

Chapter 2

Prototyping Process

The prototype narrows the WR1 concept into one playable battle slice. WR2 therefore documents the finished demo state rather than a broader future design, and it keeps partial or deferred items explicit.

2.1 Prototype

Figure 2.1 shows a retrospective low-fidelity wireframe rebuilt from the WR1 scope and the current scene layout. It remains useful because it clarifies the intended information zones: turn state, hand controls, status feedback, and the board.

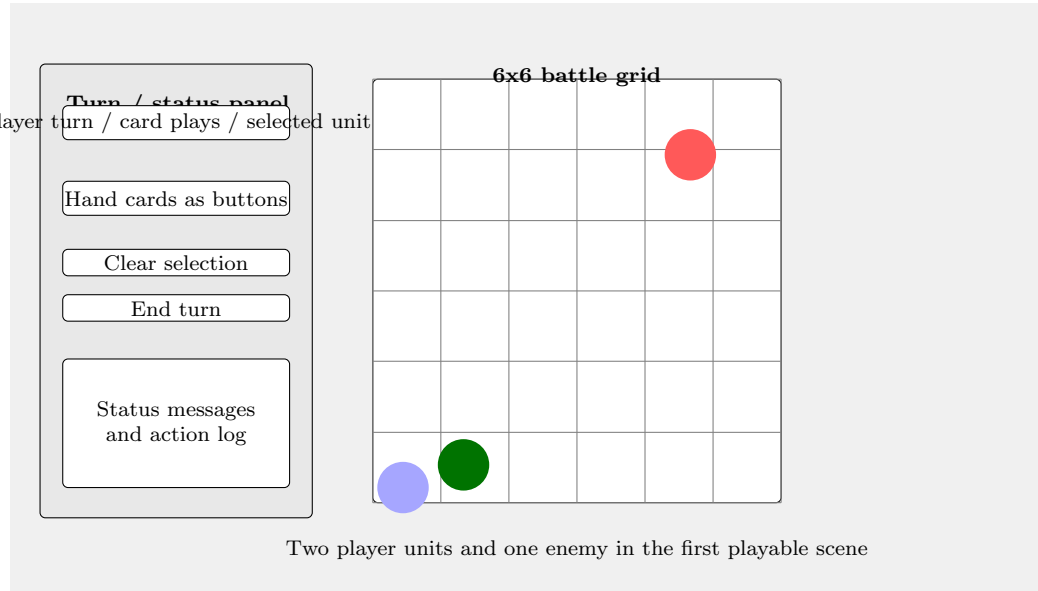

Figure 2.1: Retrospective low-fidelity wireframe reconstructed from WR1 scope and the current prototype layout

The high-fidelity evidence comes from the working Unity scene. Figure 2.2 is the main reference for the full battle view, Figure 2.3 isolates the starter hand

and top bar, and Figure 2.4 isolates the board layout for readers unfamiliar with tactical games.

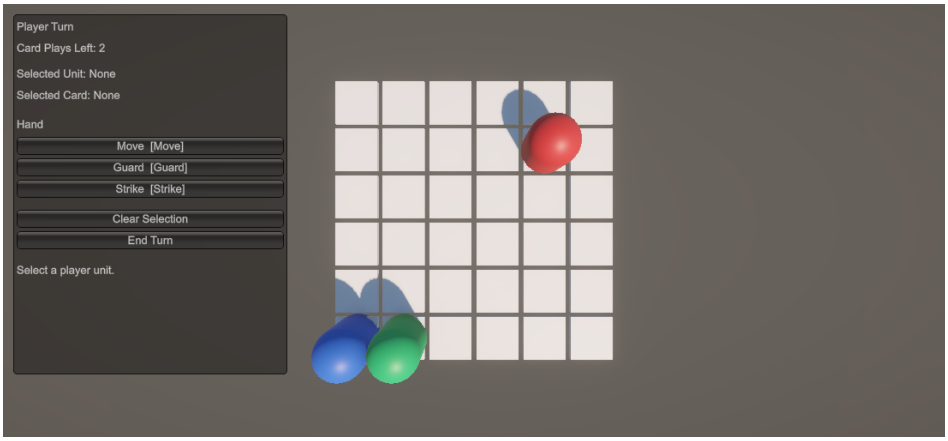

Figure 2.2: Current high-fidelity prototype with battle UI, starter hand, and 6x6 board

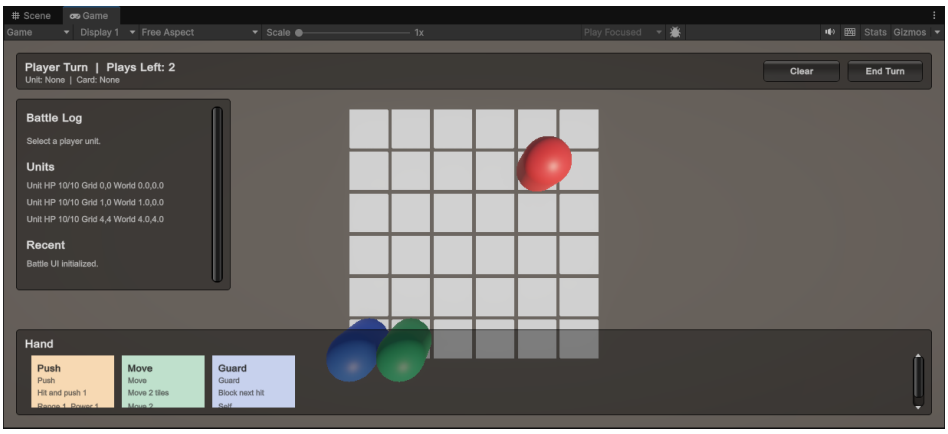

Figure 2.3: Playable hand cards and turn-state UI in the current prototype scene

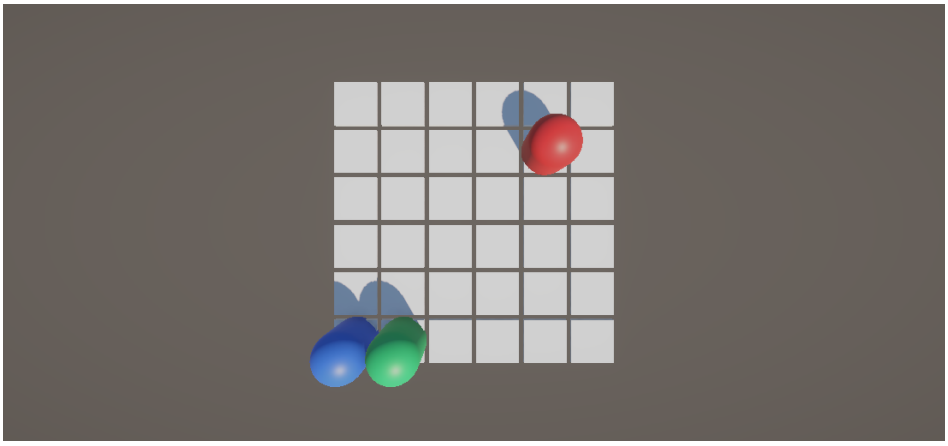

Figure 2.4: Board-only view of the current playable battle scene

2.1.1 Gameplay and Feature Walkthrough

The prototype and feature set can be explained in a direct reader-facing way as follows:

1. **What the player sees.** At battle start, the player sees a small 6x6 board with friendly and enemy units already placed, a left-side battle panel, a top-bar turn summary, and a hand of command cards. Figures 2.2 and 2.4 show this full scene composition.
2. **How one turn works.** The player selects a friendly unit, clicks one card from hand, and then clicks a highlighted tile or valid enemy target. This can be repeated until the turn's fixed card-play allowance is exhausted. The player then ends the turn and the enemy performs a simplified response. Figure 2.2 shows the board and UI in the same frame so this sequence is understandable.
3. **What the four starter cards mean.** `Move` repositions the selected unit onto a highlighted tile, `Strike` deals direct damage to a valid enemy in range, `Guard` reduces the next incoming damage to the selected unit by 1, and `Push` deals light damage and attempts to knock the target back by one tile. Figure 2.3 shows the hand zone used to trigger these four actions.
4. **What feedback the system gives.** The prototype highlights legal tiles, keeps the selected unit visible on the board, updates turn and status text, records actions in the log, and reports victory or defeat through lightweight text feedback. Figures 2.2 and 2.3 are the main references for these feedback features.
5. **Unfinished features.** The current build still lacks a stronger tutorial, a highlight legend, richer battle-end presentation, broader automated coverage, and more expressive enemy tactics. Figure 2.4 makes that absence visible because the board is shown without extra onboarding overlays.

Current feature state:

- **Implemented.** 6x6 battle grid, unit selection, starter hand and card draw, `Move`/`Strike`/`Guard`/`Push`, simplified enemy response, board highlights, and battle-end text feedback.
- **Partial.** The UI prompts the player, but stronger onboarding and clear highlight explanations are still missing.
- **Initial automated verification.** A small EditMode suite checks three battle-end scenarios in the turn manager.
- **Deferred.** Broader automated coverage, stronger playtesting, and richer audiovisual result presentation remain for the next cycle.

Main implementation constraints:

- The UI is a hybrid prototype solution that favours quick wiring and readable reporting over long-term extensibility.
- Input uses the current Input System plus physics raycasts, linking button-based card choice to tile and unit selection with little extra infrastructure.
- Enemy behaviour is deterministic so debugging, validation, and reporting

stay manageable.

- The prototype prioritises the battle loop and feedback model first; stronger presentation and broader testing are later work.

2.2 Design Validation

WR2 validation used an internal heuristic walkthrough of the frozen prototype snapshot on 01/04/2026. This stage therefore uses structured scenario review, which matches the seminar distinction between early validation and the fuller testing structure expected in WR3 [3]. The walkthrough checked three scenarios aligned with the current demo goals: selecting a player unit, playing a card against a valid target, and ending the turn to observe enemy behaviour. Appendix A.4 lists the walkthrough record and related supporting evidence. After that walkthrough, one narrow runtime fix and an EditMode test pass were added to correct the battle-end accuracy gap while keeping the prototype scope stable.

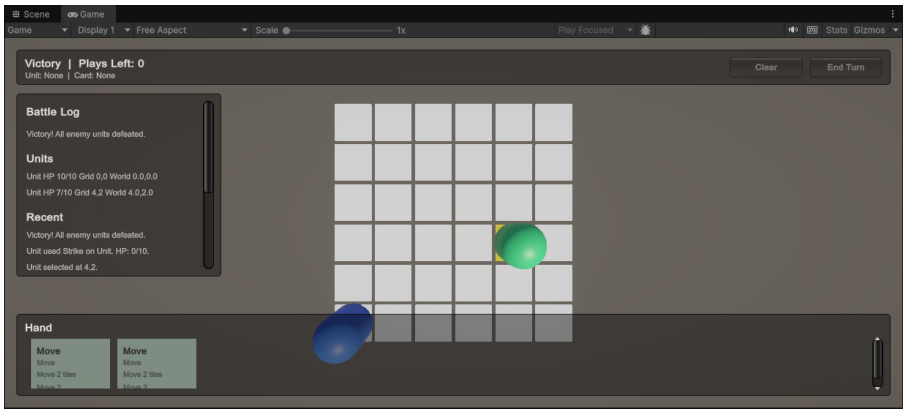

Figure 2.5: EditMode smoke test result recorded after the battle-end accuracy fix

Figure 2.5 adds an automated check alongside the walkthrough evidence. It complements the WR2 walkthrough and shows that the corrected battle-end behaviour was executed successfully in the Unity Test Framework.

The key validation findings are:

1. **Scenario: first interaction and unit selection.** Finding: the player can infer the next step from the left-side status prompt, but there is no stronger onboarding cue on the board itself. Severity: medium. Proposed action: add a clearer first-turn prompt or highlight the expected first click. Status: planned.
2. **Scenario: play a card and confirm its target.** Finding: card meanings are readable, and the board still lacks a direct explanation of move versus attack highlight colours. Severity: medium. Proposed action: add a tiny legend or one-line prompt when a card is selected. Status: planned.
3. **Scenario: end turn and observe enemy response.** Finding: enemy actions were readable through summary text, but the battle loop initially

lacked a clear end-state gate and outcome cue. Severity: high. Proposed action: add explicit battle-end termination and visible victory/defeat feedback before final WR2 closeout. Status: applied in narrow form through the prototype-accuracy fix; stronger post-action emphasis remains planned.

These findings shaped the final wording of WR2. The report presents a working vertical slice with battle-end gating, text outcome feedback, and an initial automated check. Tutorial flow, result-screen polish, and full audiovisual combat feedback remain later work.

Chapter 3

Implementation Strategy

The implementation strategy follows one principle: keep the prototype readable enough that each major gameplay responsibility can be traced to a small number of Unity scripts. This section focuses on code structure and the main trade-offs behind the current prototype.

3.1 Code Structure

At repository level, the project is split so that the Unity implementation, report sources, and evidence records stay in clearly separated areas. The simplified WR2-relevant top-level layout used during report closeout is:

```
Unity-Project
|-- coursework
|-- artifacts/uml
`-- doc/src
    |-- project-records
    |-- wr1-latex
    `-- wr2-latex
```

This layout keeps the playable Unity project inside `coursework` and stores diagrams and report evidence separately. Within that structure, the current gameplay scripts are organised into a small set of folders under `coursework/Assets/Scripts`. The directory tree below reflects the runtime structure used by the prototype.

```
Assets/Scripts
|-- Cards
|   |-- CardData.cs
|   |-- CardResolver.cs
|   `-- DeckManager.cs
|-- Common
|   `-- GameEnums.cs
|-- Core
|   |-- GameRoot.cs
|   `-- TurnManager.cs
|-- Grid
|   |-- GridManager.cs
```

```
|   `-- TileView.cs
|-- UI
|   |-- BattleUI.cs
|   `-- CardButtonView.cs
`-- Units
    |-- EnemyAI.cs
    `-- UnitController.cs
```

This structure mirrors current responsibilities. The `Core` folder owns scene bootstrapping and turn flow. `Grid` owns tile generation, occupancy, and range queries. `Cards` owns card data, deck state, and explicit card resolution. `Units` owns health, team, and enemy decision logic. `UI` connects player input to runtime state.

A file-tree view is used here because this prototype is built around a small set of scene-bound Unity `MonoBehaviour` scripts. The clearest way to communicate structure is to show where gameplay responsibilities live under `Assets/Scripts`. A traditional inheritance-heavy class diagram would suggest more architectural depth than the current prototype actually uses.

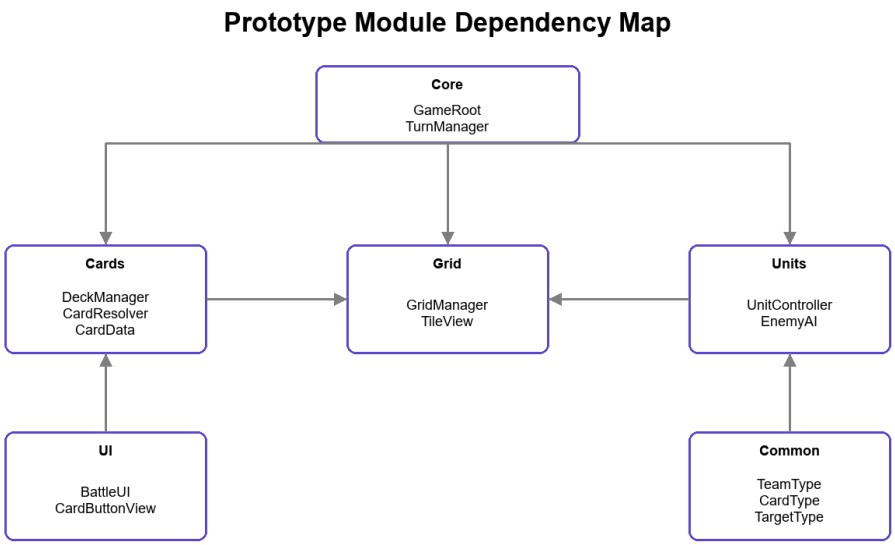

Figure 3.1: Simplified module dependency and responsibility map

Figure 3.1 focuses on code responsibility split. Dependencies flow inward toward a small number of managers, and the surrounding text explains runtime event order. This matches the goal of a small, traceable prototype.

3.1.1 Runtime Responsibilities Beyond the Diagram

The responsibility map is simplified, so the feature-level role of each main script is summarised below:

- `GameRoot`. Registers the pre-placed units in the scene, links the managers together, and provides the starting point for the playable battle slice.

- `TurnManager.` Controls player and enemy phase transitions, remaining card plays, start-of-turn draw behaviour, and the battle-end gate introduced by the prototype-accuracy fix.
- `GridManager.` Builds the 6x6 board, tracks tile occupancy, and answers movement or target-range queries used by cards and AI.
- `DeckManager.` Owns the starter deck, draw pile, discard pile, and current hand shown in the battle UI.
- `CardResolver.` Converts `Move`, `Strike`, `Guard`, and `Push` into explicit runtime effects so card behaviour stays easy to trace and test.
- `UnitController.` Stores team, hit points, guarding state, and grid position, and exposes the small set of combat operations needed by the current prototype.
- `EnemyAI.` Produces the deterministic enemy response used in WR2, keeping target choice and movement logic simple enough to observe clearly during demonstration.
- `BattleUI` **and** `CardButtonView.` Handle card selection, click routing, prompts, and log updates so the player can see the state changes caused by the core systems above.

3.2 Critical Implementation Choices

The most important implementation choices in the current prototype are:

- **Single self-contained battle.** Options considered: one battle scene versus multi-scene campaign flow. Chosen option: one battle scene, because it provides a complete vertical slice for WR2 without spending time on navigation and persistence. Trade-off: broader progression is postponed.
- **Shared unit runtime model.** Options considered: one shared controller versus separate player/enemy hierarchies. Chosen option: one shared `UnitController` so that health, guarding, movement, and grid state remain easy to trace. Trade-off: the model is less specialised than a longer-term production structure.
- **Explicit card resolver.** Options considered: `enum`-driven explicit logic versus a generic card-effect hierarchy. Chosen option: one resolver that interprets `CardType`, because the starter set is small and benefits from readable switch-based logic. Trade-off: a larger future card set would eventually need refactoring.
- **Deterministic enemy AI.** Options considered: nearest-target movement versus broader tactical AI. Chosen option: deterministic nearest-target behaviour, because it is predictable, testable, and easy to explain in WR2. Trade-off: the enemy behaviour remains strategically shallow.
- **Prototype-first UI and input path.** Options considered: a full Unity UI pipeline versus the current hybrid prototype UI with Input System and board raycasts. Chosen option: the hybrid path, because it was faster to wire into the existing scene and sufficient for WR2 reporting. Trade-off: UI

consistency and polish remain limited.

- **GitHub-first collaboration after repository cleanup.** Options considered: continue closing WR2 on a mixed local clone versus migrate report-closeout work onto a cleaned GitHub workflow with explicit PR rules. Chosen option: move current collaboration onto the cleaned GitHub repository, record the review policy, and keep earlier local/Overleaf/WeChat-era activity in supporting notes. Trade-off: current closeout work is now traceable through merged pull requests, but some earlier process evidence still relies on reconstructed notes and the task-board trail remains weaker than the repository history.

Overall, the team treated the prototype as implementation evidence for a small tactics slice. This explains why the runtime modules are explicit, why the card system stays narrow, and why some features are marked as deferred. For WR2, this keeps the implementation aligned with the report and the current demo scope.

Chapter 4

Summary and Reflections

4.1 Team Members and the Division of Labor

The implementation-phase allocation used in WR2 preparation is summarised below. It updates the earlier WR1 proposal roles and reflects the work packages needed to turn the concept into a working prototype and report pack.

- Guangjun HU (20511787): WR2 integration, requirements validation, appendix consolidation, and document traceability.
- Hongshuo HU (20808603): Unity architecture, scene wiring, core battle systems, and runtime integration.
- Cheng YE (20513323): UML and figure refinement, prototype presentation material, and supporting chapter drafting.
- Zehan CHEN (20807750): validation records, testing and meeting templates, and supporting documentation/website tasks.

4.2 Completed Process

By the end of WR2, the project had moved from proposal-level design to a stable prototype snapshot. The team narrowed the scope to one evidencable battle slice, aligned the report with the live runtime state, and corrected the battle-end accuracy gap with a small EditMode test suite around the turn manager.

WR2 also closed a documentation gap. Meeting notes, process evidence notes, decision logs, validation notes, testing records, and merged GitHub PR history now form the minimum reusable record set for WR3 and the final report.

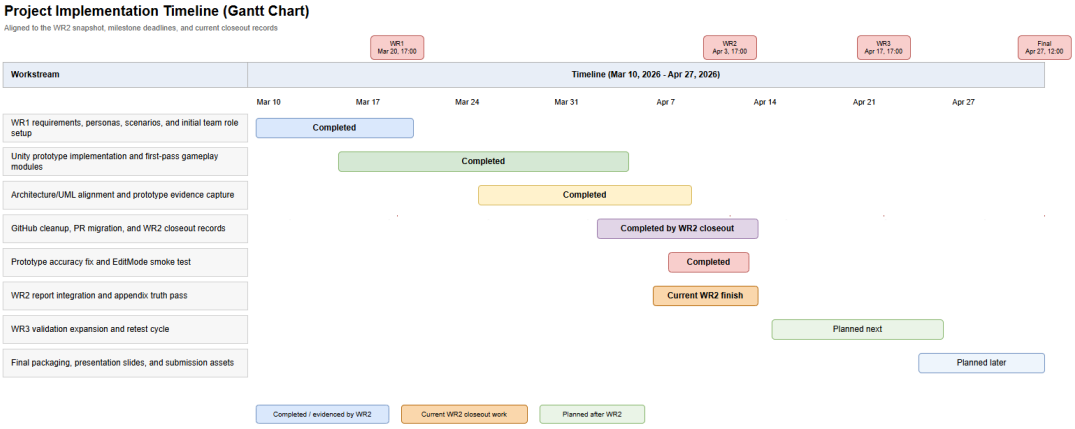

Figure 4.1: Implementation timeline aligned to coursework milestones and the current WR2 closeout snapshot

Figure 4.1 summarises the timeline from the WR1 foundation through the current WR2 closeout and the planned WR3/final-report stages. The chart separates work already completed in WR2 from work scheduled after the WR2 deadline.

4.3 Prioritised Tasks

The priorities below combine the local WR2 todo list, supporting records, and merged GitHub PR history so the report has one explicit task view instead of scattering it across appendix notes only. Short source names are used here for readability; Appendix A.4 lists the exact paths.

4.3.1 P0 Submission-Critical Tasks

P0 work fell into two concrete groups: prototype and scope accuracy, then report-package closeout on the clean GitHub workflow.

Task	Priority	Status	Evidence source	Related PR / record
Lock the prototype to a single-battle vertical slice.	P0	Completed	WR2 freeze note; architecture constraints; current scene snapshot.	Freeze note; decision D-01.
Implement and evidence the four starter cards.	P0	Completed	Snapshot verification; runtime card resolution; prototype screenshots.	Snapshot verification; architecture constraints note.
Implement the player turn loop, draw/discard flow, and enemy response.	P0	Completed	Current runtime; screenshot evidence; walkthrough.	Snapshot verification; walkthrough note.
Fix battle-end accuracy so the loop stops cleanly and reports an outcome.	P0	Completed	EditMode verification note; current battle-end behaviour.	PR #5; prototype accuracy note.
Align WR2 baseline and scope-control records with the clean repository line.	P0	Completed	Baseline records imported into the current source tree.	PR #6; WR2 todo; asset register.
Import the WR2 LaTeX report package into the clean GitHub workflow.	P0	Completed	Tracked WR2 source tree and compiled report package.	PR #7; WR2 LaTeX source tree.

Task	Priority	Status	Evidence source	Related PR / record
Add collaboration history and support evidence notes for the closeout workflow.	P0	Completed	Process notes and recovered collaboration-history package.	PR #8; collaboration history note.
Import the missing WR2 supporting records used by the report body.	P0	Completed	Validation, testing, decision, and process record files now present in the repo.	PR #9; project-records import.
Refresh WR2 wording so the body matches the merged prototype state.	P0	Completed	Updated report sections and rebuilt PDF.	PR #12; current WR2 section files.
Run a truth pass on records, appendix wording, and supporting claims.	P0	Completed	Updated appendix, refreshed supporting notes, and snapshot verification wording.	PR #13; appendix and records truth pass.
Close out figures and references so the final report package is internally consistent.	P0	Completed	Current tracked media, figure sources, and rebuilt PDF.	PR #15; figure/reference closeout pack.

4.3.2 P1 Strongly Recommended Tasks

Task	Priority	Status	Evidence source	Related PR / record
Run one final diagram-to-code cross-check before submission packaging.	P1	Open	Snapshot verification already covers the main runtime modules, but a final pass remains useful.	Background PR #1; snapshot verification; WR2 todo.
Do a final proofread by someone who did not draft the section.	P1	Pending	WR2 todo list and review checklist both keep this open.	WR2 todo; review sync checklist.
Review evidence consistency one more time before export.	P1	Pending	Current records make the check possible, but the sign-off task is still open.	WR2 todo; asset register.
Import optional stronger repository screenshots for appendix support.	P1	Optional strengthening	GitHub history is already visible, but screenshots would improve presentation quality.	Canonical repo evidence note.
Improve highlight explanation and first-turn onboarding cues.	P1	Planned, not yet applied	Walkthrough findings identify this as a medium-severity usability gap.	Walkthrough note.

4.3.3 P2 Deferred or Externally Blocked Tasks

Task	Priority	Status	Evidence source	Related PR / record
Import historical task-board captures from the real board tool.	P2	Blocked externally	The current repository only preserves a retrospective reconstruction, not original board evidence.	Task-board evidence note; Figure A.1.
Replace recovered attendance notes with stronger confirmed meeting evidence.	P2	Blocked externally	Attendance evidence remains weaker than the repository trail.	Collaboration history; recovered checkpoint note.
Run stronger team playtesting beyond the internal heuristic walkthrough.	P2	Deferred to WR3 / final report	WR2 currently uses an internal walkthrough only.	Walkthrough note; WR3 prep log.
Expand automated coverage beyond the battle-end fix.	P2	Deferred to WR3 / final report	Only the narrow battle-end EditMode suite is currently evidenced.	PR #5; prototype accuracy note.
Add a richer result screen and stronger end-state presentation.	P2	Deferred	WR2 only claims lightweight text outcome feedback.	Walkthrough note; PR #5.
Add richer audiovisual combat feedback and stronger enemy expressiveness.	P2	Deferred	Current WR2 snapshot intentionally keeps feedback and AI behaviour lightweight.	Architecture constraints; implementation decisions.

4.4 Future Plan

The next stage connects directly to WR3 and the final report:

- strengthen battle-end presentation beyond the current top-bar/log feedback and tighten enemy-turn emphasis;
- improve onboarding cues for first-time interaction with the board and card selection flow;
- supplement the now-active GitHub PR history with any stronger task-board captures or confirmed attendance evidence the team can still recover;
- convert the current validation notes into a fuller WR3 testing pack with test cases, bug tracking, and retest records;
- expand the current automated coverage beyond battle-end behaviour into stable rule-heavy systems such as grid range checks and card resolution rules.

In short, WR2 has produced a stable prototype foundation and a clearer list of follow-up work for WR3.

Bibliography

[1] University of Nottingham. Comp4135 coursework 1 specification, 2026. Local course specification used for WR2 scope, deadlines, and weekly report expectations.

[2] University of Nottingham. Comp4135 seminar 5: Prototyping, 2026. Seminar material on prototype goals, fidelity, and requirement validation.

[3] University of Nottingham. Comp4135 seminar 6: Software validation, 2026. Seminar material on validation, implementation design, and testing preparation.

[4] University of Nottingham. Comp4135 weekly report 2 template, 2026. Moodle WR2 template PDF used to recover the correct Weekly Report 2 chapter structure.

Appendix A

Meeting Minutes and Working Records

A.1 Recovered Checkpoint (15/03/2026)

No formal implementation meeting minutes were archived in the local workspace for the first prototype checkpoint. This recovered note summarises the result of an early classroom discussion and follow-up implementation alignment that informed the first prototype slice. The in-class discussion itself has no formal record, so the checkpoint summary was compiled from dated artifacts that are present in the repository: the architecture constraints note, the first-pass script plan, and the prototype screenshots created on 15/03/2026.

- **Record type:** recovered implementation checkpoint compiled from local artifacts after an unrecorded classroom discussion.
- **Date:** 15/03/2026.
- **Attendance:** exact attendance was not archived in the local workspace and should be replaced with confirmed team attendance if chat or calendar records are available.
- **Sources used:** the architecture note and first-pass script plan listed later in the Supporting Record Index, plus prototype screenshots timestamped on 15/03/2026.
- **Main focus:** reduce the original game concept to one battle scene, four starter cards, a readable control loop, and a minimum set of scripts that can compile and run inside the existing Unity project.
- **Confirmed decisions:** single-battle scope, shared unit controller, deterministic enemy response, explicit card types, button-driven card interaction, and a WR2 scope without campaign or deckbuilding systems.
- **Follow-up actions:** finish the first gameplay skeleton, refresh UML diagrams to match implementation, and gather prototype evidence for WR2.

A.2 WR1 Kickoff and Alignment Meetings

The following two short records are retained because WR2 directly inherits the role allocation and initial scope decisions set during WR1 preparation.

A.2.1 Meeting 1 (17/03/2026)

Time

17/03/2026

Attendance

- Guangjun HU
- Hongshuo HU
- Zehan CHEN

Context

Team members shared their initial game ideas and implementation interests, then narrowed the coursework direction to a tactical card battle demo. The meeting also established the first division of labour and the expected delivery dates for WR1 inputs.

Decisions

Project direction was confirmed as Tactical Cards, the initial role allocation was agreed, and individual task expectations were set for the first weekly cycle.

Owner

Guangjun HU, Hongshuo HU, and Zehan CHEN, with follow-up input from Cheng YE.

Deadline

18/03/2026 to 19/03/2026 for the first WR1 materials and task allocation outputs.

Follow-up

These decisions fed directly into the project direction, division of labour, and requirement sections of WR1.

A.2.2 Meeting 2 (18/03/2026)

Time

18/03/2026

Attendance

- Guangjun HU
- Cheng YE

Context

The team aligned the current requirement scope around a tactical card battle demo, confirmed UML-related responsibilities, and standardised a lightweight meeting-minutes format so later weekly records would be more traceable than the earliest prototype cycle.

Decisions

The current WR1 scope was aligned around a tactical card battle demo, UML diagram production was assigned, and a unified meeting-minutes format was defined for subsequent meetings.

Owner

Guangjun HU and Cheng YE, with follow-up support from the whole team.

Deadline

19/03/2026 for WR1 alignment, followed by use of the agreed meeting-minutes format in later weekly cycles.

Follow-up

These outcomes were reflected in WR1 requirements, UML diagrams, and the documentation process used by later reports.

A.3 WR2 Evidence Freeze Note

The WR2 documentation pack uses an evidence freeze line of **02/04/2026 12:00**. This freeze is recorded in `wr2-snapshot-freeze-2026-04-02.md`. After that point, only changes that materially improve submission quality should be described in WR2. New features developed later should be documented in WR3 or the final report instead of being backfilled into this report.

A.4 Supporting Record Index

The file set referenced across WR2 is summarised below. It combines upstream design files with later supporting records so the cited materials can still be located clearly from the repository layout.

- `doc/src/wr1-latex/main.tex`: WR1 report source containing the original requirement, scenario, and system-model baseline inherited by WR2.
- `doc/src/onepager_en.md`: early internal concept note used before the prototype scope was narrowed.
- `doc/src/demo_architecture_constraints.md`: internal architecture-constraint note used to cut the concept down to one battle scene.
- `doc/src/demo_first_pass_scripts.md`: first-pass gameplay script plan for turning the narrowed design into a playable Unity slice.
- `doc/src/project-records/wr2-snapshot-freeze-2026-04-02.md`: freeze note defining the prototype and evidence snapshot used for WR2.
- `prototype-snapshot-verification-2026-04-02.md`: runtime-to-document alignment record used by the Chapter 1 traceability table and the Chapter 4 task priorities.
- `wr2-todo.md`: explicit priority list used as the baseline for the Chapter 4 P0/P1/P2 task summary.
- `wr2-asset-register.md`: reusable evidence list, missing items, suggested owners, and due times.
- `process/canonical-repo-evidence-2026-04-02.md`: canonical repository evidence note, historical discovery summary, and the current recommendation for using merged GitHub history as repo-workflow evidence.

- `process/task-board-evidence-2026-04-02.md`: task-board evidence note, discovery summary, and import checklist.
- `process/project-implementation-timeline-gantt.drawio`: draw.io source for the coursework timeline Gantt chart used in the WR2 reflections section.
- `process/task-board/retrospective-wr2-closeout-board.png`: retrospective coordination board reconstructed from the recorded TODO list and merged PR history for explanation only.
- `process/discovery/README.md`: command-level discovery artifacts explaining what was checked from this workspace.
- `meetings/meeting-minutes-template.md`: lightweight meeting template for future weekly records.
- `meetings/meeting-2026-03-17-wr1-kickoff.md`: preserved WR1 kickoff note covering project direction and first role allocation.
- `meetings/meeting-2026-03-18-wr1-alignment.md`: preserved WR1 alignment note covering scope confirmation and meeting-template setup.
- `meetings/implementation-checkpoint-recovered-2026-03-15.md`: transparent recovered note based on local artifacts.
- `decisions/wr2-implementation-decisions.md`: decision log extracted from the current implementation direction.
- `collaboration-history-2026-04-02.md`: concise explanation of the move from early WeChat/Overleaf coordination to the current GitHub PR workflow.
- `validation/wr2-heuristic-walkthrough-2026-04-01.md`: internal WR2 walkthrough and findings.
- `testing/prototype-accuracy-editmode-run-2026-04-02.md`: EditMode verification note for the battle-end accuracy fix.
- `testing/wr3-prep-log-2026-04-01.md`: initial test and bug tracking preparation note for the next report cycle.
- Merged GitHub PRs #5, #6, #7, #8, #9, #12, #13, and #15 in the project repository: remote evidence used by the Chapter 4 task table and the workflow traceability discussion.

These records are intentionally lightweight. Their purpose is to stop evidence loss between WR2 and WR3 and to preserve the material that already exists from the prototype cycle.

A.5 Traceability and Priority Sources

The new WR1 to WR2 traceability table in Chapter 1 is derived mainly from the WR1 source, the internal architecture and first-pass script notes, the prototype snapshot verification record, the walkthrough findings, and the WR2 implementation decision log. The prioritised task tables in Chapter 4 combine the WR2 todo list, the evidence register, workflow/process notes, and the merged GitHub PR history listed above. This appendix keeps those source pointers explicit so the new tables do not rely on unspoken evidence.

A.6 Retrospective Coordination Board

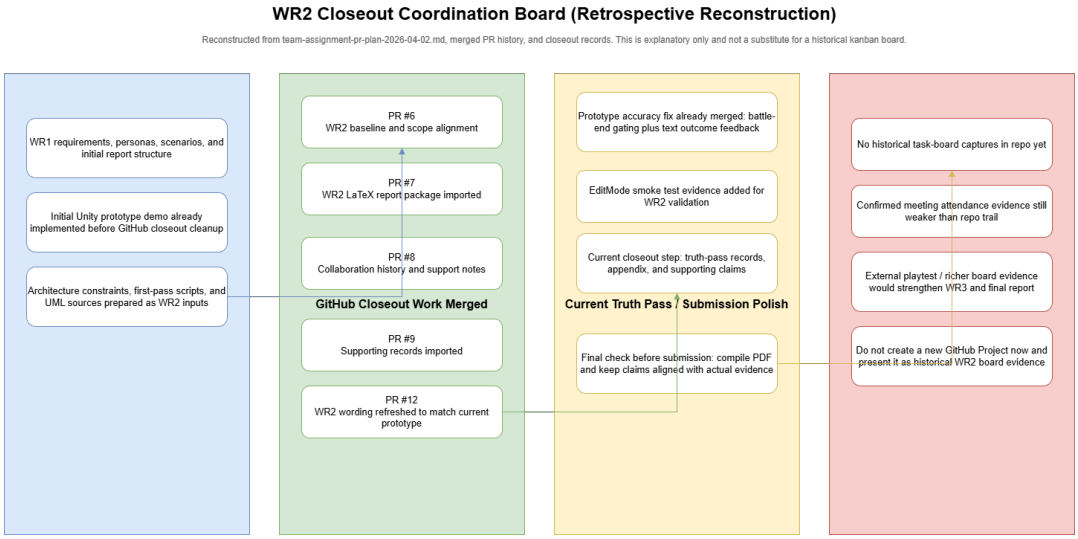

Figure A.1: Retrospective reconstruction of the WR2 closeout board, based on recorded TODO lists and merged GitHub PR history

Figure A.1 is included as an explanatory reconstruction because no historical task-board capture survives in the repository. It helps summarise how the closeout work was grouped after the move to GitHub PR workflow. The figure is presented as a retrospective explanation of the closeout stage.